

OSS Pulse: How Coding Agents Reshaped Open-Source Development

Tracking 1.46 Billion GitHub Events Across the 2022–2026 Agent Era

Jhe Chen Li (jl17797) Bo Yu (by2566)
Group 9 — BDAD Spring 2026, New York University

May 2026

Abstract

We present **OSS Pulse**, a large-scale empirical study of open-source software ecosystem health across the 2022–2026 coding agent era. By combining 1.46 billion GitHub Archive events (five quarterly snapshots), 2.8 million Hugging Face Hub model records, and PyPI download statistics for 46 AI/ML libraries, we construct a multi-dimensional health framework that goes beyond traditional star-based metrics. Our analysis reveals three structural shifts: (1) development became *lighter*—51% more developers but 42% fewer contributors per repository; (2) the weekend activity gap closed—weekend share rose from 25.8% to 29.7%, consistent with (though not proof of) round-the-clock automation; and (3) high-frequency push activity exhibited a two-phase pattern—explosive growth in 2025 followed by normalization in 2026, plausibly tracking but not equivalent to coding-agent adoption. We processed 930+ GB of raw data into 87.9 GB of cleaned Parquet files through 10 Spark analytics jobs on a Google Cloud Dataproc cluster.

1 Introduction

GitHub stars have long served as a proxy for open-source project health. However, in the era of coding agents—tools such as Cursor [2], Claude Code [1], and Devin [6] that automate large portions of the software development workflow—this proxy is breaking down. DeepSeek-R1, for instance, is among GitHub’s most-starred repositories with over 93,000 watchers, yet only 5 distinct accounts pushed code to it during 2026-Q1.

The period from 2022 to 2026 spans a remarkable

transformation in software development: from the pre-ChatGPT baseline (2022), through the LLM explosion (2023–2024), into the agent era (2025–2026). We hypothesize that coding agents are fundamentally altering open-source contribution patterns, and that traditional metrics fail to capture these changes.

Research questions.

1. How has the macro-level structure of the OSS ecosystem changed across five annual snapshots?
2. Do coding agents produce measurable changes in development rhythms?
3. Can a multi-dimensional health score better characterize project vitality than star counts alone?

Contributions.

- A longitudinal dataset of 1.46B GitHub events across five Q1 snapshots (2022–2026).
- A composite health score integrating Hugging Face adoption, PyPI downloads, and GitHub activity.
- Empirical evidence of three structural shifts in OSS development driven by coding agents.
- An open-source Spark pipeline of 10 analytics jobs.

2 Related Work

OSS health metrics. Coelho and Valente [5] characterized failure modes of modern OSS projects through commit and contributor decay; Avelino et al. [3] formalized the *truck factor* as a commit-based concentration measure. We extend the same intuition from commits to PR-level diversity, and complement it with cross-platform demand signals (Hugging Face, PyPI). The CHAOSS project [4] main-

Table 1: GitHub Archive data scale per era.

Era	Events	Repos	Actors
2022-Q1	206.8M	17.0M	7.0M
2023-Q1	247.4M	19.9M	7.8M
2024-Q1	364.4M	17.6M	8.9M
2025-Q1	376.4M	20.2M	10.2M
2026-Q1	265.9M	20.9M	10.6M
Total	1.46B	—	—

tains a catalogue of standardized community-health indicators, but most published indicator studies pre-date the agent era.

GitHub star inflation. He et al. [9] identified approximately 6 million suspected fake stars on GitHub between 2019 and 2024, with a substantial fraction of the flagged repositories subsequently removed. Our 2026-Q1 observation—a 63% year-over-year drop in WatchEvents—is *consistent with* the tail of this clean-up, although the magnitude of the drop suggests additional factors (§7).

AI-assisted development. Recent surveys document the rise of LLM-driven coding tools and agents [7], but empirical, ecosystem-wide quantitative analyses of their behavioral fingerprint on public-repository activity remain scarce. Our work fills this gap by quantifying behavioral shifts across 1.46B events.

3 Data Sources

We integrate three complementary data sources to enable multi-dimensional health assessment.

3.1 GitHub Archive

GH Archive¹ provides hourly snapshots of all public GitHub events. We collected five Q1 windows (January–March) for 2022 through 2026, plus the full 2025 calendar year as supplementary data. Table 1 summarizes the scale. Five event types are retained: `PushEvent`, `PullRequestEvent`, `WatchEvent`, `IssuesEvent`, and `ForkEvent`.

3.2 Hugging Face Hub

We captured a snapshot of the Hugging Face Hub in April 2026, comprising 2,815,064 model

¹<https://www.gharchive.org/>

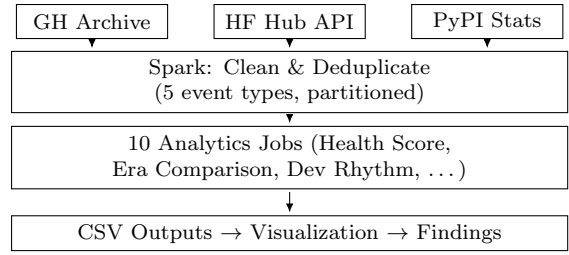


Figure 1: Pipeline architecture: three data sources feed into Spark cleaning, then fan out to 10 analytics jobs.

records. Key fields include `modelId`, `pipeline_tag`, `library_name`, `downloads` (30-day), `likes`, and `parameter_count`. The data was ingested via the `huggingface_hub` Python API and processed through the same PySpark pipeline used for GH Archive.

3.3 PyPI Download Statistics

Monthly download counts for 46 AI/ML Python libraries were obtained via Google BigQuery’s public PyPI dataset, covering all 12 months of 2025.

3.4 Seed Repository List

We curated 93 AI/ML repositories spanning 14 categories (frameworks, LLM serving, inference, agents, RAG, diffusion, training, evaluation, etc.) to enable focused AI-vs-general comparisons.

4 Pipeline Architecture

4.1 System Overview

The pipeline runs on NYU’s Google Cloud Dataproc cluster. All data is stored in HDFS as Parquet files partitioned by `event_date`.

The architecture follows a three-stage design (Figure 1):

1. **Ingest:** Raw JSON.gz files are parsed with explicit schemas, converted to Parquet with column pruning (70% storage reduction).
2. **Clean:** Deduplication by `event_id`, null filtering, timestamp normalization, and schema unification.
3. **Analyze:** 10 independent Spark jobs produce CSV outputs for downstream visualization.

Total data reduction: 930+ GB raw → 87.9 GB cleaned (90.5%).

Table 2: Spark analytics jobs.

#	Output	Description
01	repo.daily	Daily activity per AI repo
02	hf_gh_join	HF models + GH stats
03	health_score	Composite health score
04	top_repos	Top 1000 repos (ecosystem)
05	ai-vs-general	AI vs. general repos
06	star_hype	Star growth & hype
07	contributor	Bus factor, concentration
08	era_comparison	Cross-era (5 Q1 snapshots)
09	era_deep_dive	Per-repo era analysis
10	dev_rhythm	Weekend/weekday, push freq.

4.2 Schema Design

All five event types are mapped to a unified schema with 10 fields: `event_id`, `event_type`, `event_date` (partition key), `event_ts`, `actor_login`, `repo_name`, `payload_action`, `pr_number`, `pr_merged`, and `push_distinct_size`.

4.3 Analytics Jobs

Table 2 summarizes the 10 analytics jobs.

5 Data Quality

Across all five eras, the four core fields (`event_type`, `event_date`, `actor_login`, `repo_name`) have **zero nulls**. Payload-specific fields (`pr_merged`, `pr_number`, `payload_action`) are structurally null for non-applicable event types.

For the 2026-Q1 era, we re-downloaded all 2,160 raw hourly files and re-cleaned them. 580 corrupt gzip files were handled via Spark’s `ignoreCorruptFiles` option. Post-cleaning diff against the original was less than 0.39%. All eras are deduplicated by `event_id`.

6 Engineering Challenges

6.1 HDFS Quota Management

Processing five years of Q1 data required ~700 GB of raw storage, but the HDFS quota was 500 GB.

Solution 1—Rolling pipeline: Download one era’s raw data, clean to Parquet, delete raw, proceed

Table 3: GitHub API schema change.

Field	≤2025	2026+
<code>pr.merged</code>	true/false	NULL
<code>action</code>	open/close	+ “merged”
<code>push.distinct_size</code>	present	removed

to next era. Storage stays under quota.

Solution 2—Per-era processing: Compute metrics for each era independently, save intermediate results. A final merge combines only small intermediate CSVs. If YARN terminates mid-job, completed eras are preserved.

6.2 2026 Schema Breaking Change

Job 08 initially reported zero merged PRs for 2026-Q1. Investigation revealed that GitHub’s REST/Webhook event payloads changed between 2025 and 2026 [8]: the `pr.merged` field became permanently NULL, replaced by a new `payload.action="merged"` value, and the `push.distinct_size` field was removed from `PushEvent` payloads (Table 3). The latter prevents us from computing 1-commit-push ratios and average commit-size statistics for 2026-Q1.

We implemented a backward-compatible fix:

```
merged = (
    ((col("payload_action") == "closed")
     & col("pr_merged").eqNullSafe(True))
    | (col("payload_action") == "merged")
)
```

6.3 Billion-Record Shuffle Optimization

Computing top-1000 repos across 1B+ events required a full cross-partition shuffle (72 GB → 83 GB shuffle, >3 hours). We tuned `shuffle.partitions` (400→200), cached `DataFrames`, and downstream jobs read pre-aggregated outputs. **Retrospective:** Bucketing by `repo_name` at ingest would eliminate the shuffle entirely.

7 Findings

7.1 Finding 1: Five-Year Ecosystem Shift

Figure 2 visualizes the ecosystem evolution; Table 4 presents the key metrics.

More developers are participating, but each repo receives fewer contributors. Pushes are smaller and

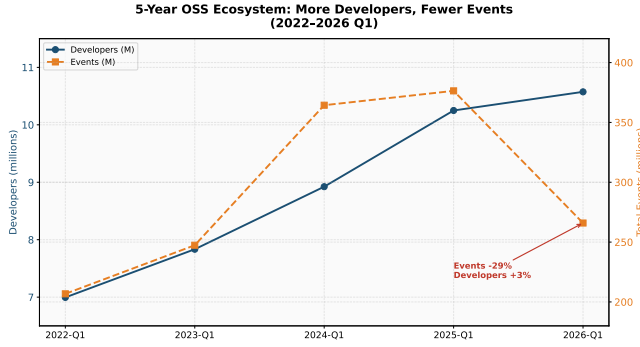


Figure 2: Five-year ecosystem shift across 2022–2026 Q1 snapshots. Developers grew 51% while contributors per repo fell 42%.

Table 4: Ecosystem evolution: 2022 vs. 2026 Q1. [†]1-commit push % reports the latest era for which `push.distinct_size` is available (2025), since GitHub removed this field in 2026 (§6).

Metric	2022	2026	Δ
Developers	7.0M	10.6M	+51%
Total events	207M	266M	+29%
1-commit push % [†]	87%	94% (2025)	+7pp
PR/push ratio	0.098	0.082	−16%
Contrib./repo	4.88	2.84	−42%

more frequent, while the PR/push ratio declined 16%—less ceremony, more direct pushes.

The 2026 event drop. Total events declined 29% from 2025 to 2026. The drop is concentrated in popularity signals (WatchEvents −63%, ForkEvents −67%) rather than in engineering activity (PushEvents −28%). *PushEvent’s share rose from 83% in 2025 to 85% in 2026 (versus 71% in 2022).* This pattern is consistent with the tail of GitHub’s clean-up of suspected fake-star accounts reported by [9], but several alternative or complementary explanations cannot be ruled out: (i) the magnitude of the WatchEvent drop (−12.4M in a single quarter) exceeds what a one-time removal of ~6M cumulative fake stars (2019–2024) can account for; (ii) the 67% ForkEvent decline is unlikely to be driven by fake-star clean-up, since forks are not the focus of that work; (iii) the GitHub event-payload schema change of late 2025 (§6) altered how some interactions are emitted; and (iv) 580 corrupt hourly archives in 2026-Q1 were skipped via Spark’s `ignoreCorruptFiles`, with an estimated cleaning-stage delta of $\leq 0.39\%$ relative to a re-download (§5).

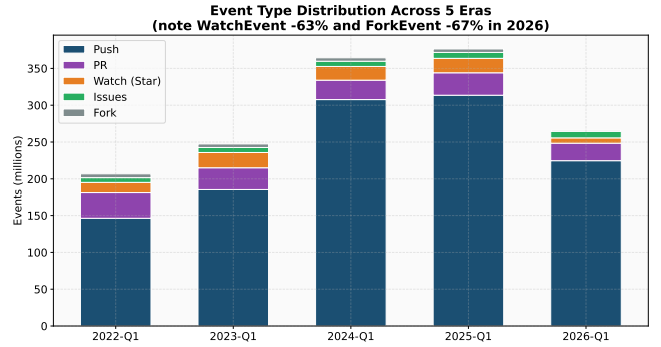


Figure 3: Event type composition across five eras. Push dominance grows as popularity signals (Watch, Fork) decline sharply in 2026.

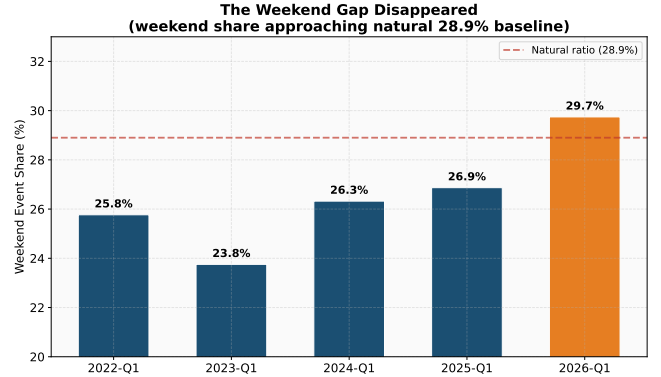


Figure 4: Weekend event share across five eras. By 2026, weekend activity exceeds the 28.9% natural baseline—a pattern compatible with round-the-clock automation, though human-side factors (globalization, on-call pressure) cannot be excluded.

We therefore present this drop as an empirical observation rather than as evidence of any single cause.

7.2 Finding 2: The Weekend Gap Disappeared

In a natural weekly distribution, weekends account for 28.9% of days (2/7). Human developers historically produce a weekend gap. Figure 4 shows this gap vanishing by 2026.

The closure is driven primarily by **PushEvent**, whose weekend ratio rose monotonically from 24.8% (2022) to 29.4% (2026). The **PullRequestEvent** weekend ratio is U-shaped over the same period (31.6% → 18.4% → 29.5%), reflecting volatile bot-driven activity (Dependabot/Renovate `synchronize` events) rather than a uniform secular trend; we therefore anchor this finding on Push. By

Table 5: Weekend event share across five eras.

Era	Wknd %	vs. 28.9%
2022-Q1	25.8%	−3.1pp
2023-Q1	23.7%	−5.2pp
2024-Q1	26.3%	−2.6pp
2025-Q1	26.9%	−2.0pp
2026-Q1	29.7%	+0.8pp

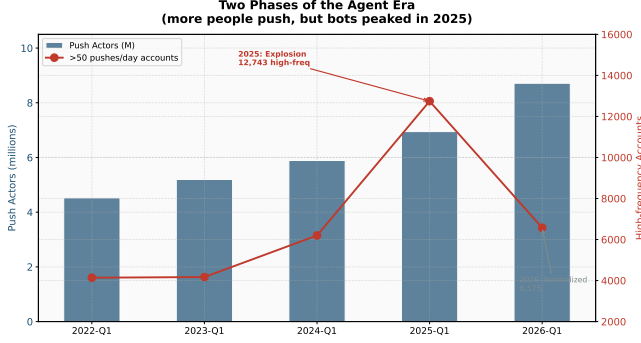


Figure 5: Two phases of high-frequency push activity: 2025 explosion (the >1K-pushes cohort nearly doubled) followed by 2026 normalization.

2026, push activity shows essentially no weekly periodicity. This is *consistent with* (but not proof of) automated agents operating around the clock; alternative drivers—broader geographic participation, increased CI/CD automation, and weekend on-call work—would yield a similar signature and cannot be separated without actor-level labels.

7.3 Finding 3: Two Phases of High-Frequency Push Activity

We track *high-frequency push accounts* (>1,000 pushes per quarter, or >50 pushes per day on the busiest day) as a proxy for automation-assisted activity. We caution that this cohort plausibly mixes coding agents, CI/build bots (e.g., the top pusher on `tensorflow/tensorflow` is a CI bot, see §7.4), scrapers, and a small number of human power users; the proxy therefore over-counts true agent adopters. With that caveat, the cohort exhibits a distinctive two-phase pattern (Figure 5, Table 6).

Phase 1—Explosion (2025): The >1K-pushes cohort nearly doubled (5,347→9,788) and the >50-pushes-per-day cohort more than doubled. This is the kind of footprint expected from early automation adopters running coding agents and bots at super-human rates, although individual-level confirmation

Table 6: Push actor statistics (2024–2026).

Metric	'24	'25	'26
Push actors	5.89M	6.95M	8.71M
>1K pushes	5,347	9,788	6,351
>50/day	6,197	12,743	6,575
Avg push/actor	52.2	45.1	25.8
Median pushes	6	6	4

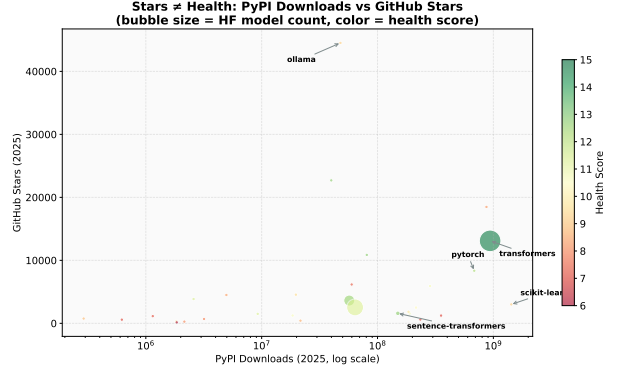


Figure 6: Stars vs. health score for AI repositories. High stars do not guarantee high health (e.g., Ollama).

is not possible from public event metadata alone.

Phase 2—Normalization (2026): The high-frequency cohort dropped −35%—a movement plausibly driven by GitHub’s tightening of automation policies and bot clean-up—while total push actors grew to 8.71M (+25%) and average pushes/actor fell 43%. The aggregate signature: *more people push, but each pushes less*.

7.4 Finding 4: Stars $\neq$ Health

Our composite health score reveals that star counts are weak predictors of project health:

$$H = 0.30 S_{HF} + 0.20 S_{PyPI} + 0.15 S_{\star} + 0.15 S_{push} + 0.10 S_{PR} + 0.10 S_{days} \quad (1)$$

where each S is a min–max normalized score on the AI-repo subset. Across the 33 AI repos with complete metadata, the Spearman rank correlation between GitHub stars and H is only $\rho = 0.46$ (Pearson $r = 0.24$); only 4 of the top-10-by-stars repos appear in the top-10-by- H . Stars and downstream usage diverge substantially.

Figure 6 and Table 7 illustrate how three distinct archetypes emerge.

Table 7: Three health archetypes.

Archetype	Example	Stars	Usage Signal
True leader	transformers	13K	PyPI 938M, HF 835K
Quiet power	sent.-trans.	1.6K	HF 519M downloads
Eco. outsider	ollama	44.5K	PyPI 47.8M, HF 241

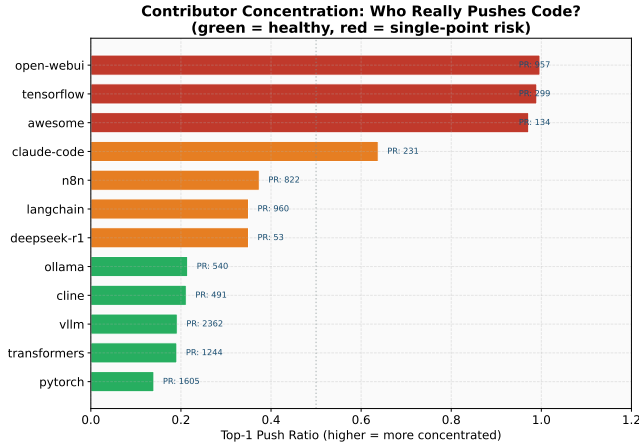


Figure 7: Contributor concentration across top repositories. Red indicates high bus-factor risk.

Sentence-transformers has 1,588 stars but 519M HF downloads—invisible to star-based rankings. Ollama has 44.5K stars but only 241 HF downloads, because it uses its own model registry. No single metric suffices; triangulation is essential.

7.5 Finding 5: Contributor Concentration

AI repos exhibit systematically higher contributor concentration (Figure 7).

Key examples:

- **open-webui**: `top1_push_ratio` = 0.997 (one person, 99.7% of pushes), yet 957 PR contributors.
- **DeepSeek-R1**: 93,947 watchers, 5 push contributors.
- **pytorch**: `top1_push_ratio` = 0.14, 1,605 PR contributors—healthy distribution.

As noted in §7, top pushers are sometimes CI bots (e.g., TensorFlow): push concentration must always be cross-validated with PR-author diversity before drawing bus-factor conclusions.

8 Discussion

Implications for OSS health measurement.

The “GitHub stars” metric is becoming actively misleading in the agent era. As agents generate automated activity, the signal-to-noise ratio of any single metric degrades. Multi-dimensional frameworks—combining community engagement (HF downloads), demand signals (PyPI installs), and engineering intensity (PRs, push diversity)—are necessary.

The “lighter development” paradox. More developers than ever (10.6M, +51%), yet per-repo contributor counts dropped 42%. One plausible reading is that automation lets individuals touch more projects simultaneously but with shallower per-project engagement; equally plausible is that the long tail of single-commit accounts grew without a matching growth in core maintainers. We cannot adjudicate between these from event-level data alone.

24/7 development and sustainability. The vanished weekend gap raises sustainability questions, but it does not by itself indicate whether the additional weekend activity is automated or human. Distinguishing human from automated event sources—through commit-message heuristics, signed-commit metadata, or actor-name patterns—is critical for future work.

Limitations and threats to validity.

- *External*: Q1-only sampling may miss seasonal effects; the 93 seed repos may not represent the full AI ecosystem.
- *Construct*: we cannot reliably distinguish agent-generated from human-generated events at the individual level, so “high-frequency push activity” is at best a proxy for agent adoption.
- *Internal*: the 2026 event drop conflates the tail of GitHub’s fake-star clean-up [9], the late-2025 schema change [8], and possible genuine behavioral changes; we cannot decompose the contributions.
- *Construct*: health-score weights are heuristic; a sensitivity analysis (e.g., equal weights or doubling S_{PyPI}) is left to future work but is not expected to overturn the qualitative ranking divergence reported in §7.4, given $\rho = 0.46$ between stars and H .
- *Internal*: 580 corrupt 2026-Q1 hourly archives were skipped via `ignoreCorruptFiles`, contributing $\leq 0.39\%$ to event counts.

9 Lessons Learned

1. **Design for the constraint.** The 500 GB quota forced a rolling pipeline that proved more crash-safe than a monolithic approach.
2. **Never trust a live API schema.** The 2026 change silently broke our merge query—always assert assumptions.
3. **Validate anomalies externally.** The 2026 event drop initially looked like a pipeline bug; cross-referencing the magnitude and timing with He et al.’s fake-star clean-up [9] and the 2025 GitHub event-payload schema change [8] convinced us it reflected real, multi-cause platform-side effects rather than a defect in our pipeline.

10 Conclusion

We analyzed 1.46 billion GitHub events across the 2022–2026 coding-agent era, complemented by Hugging Face and PyPI data. Five findings:

1. **Development got lighter:** +51% developers, −42% contributors per repo, smaller and more frequent pushes.
2. **Weekends flattened:** Weekend activity reached 29.7%, eliminating the historic human work–rest signal—compatible with, but not proof of, round-the-clock automation.
3. **Two-phase high-frequency push activity:** 2025 explosion (>1K-pushes cohort nearly doubled) → 2026 normalization (broader but lighter), plausibly tracking but not equivalent to coding-agent adoption.
4. **Stars $\neq$ health:** Spearman $\rho = 0.46$ between stars and our composite health score on the AI subset; only 4 of the top-10-by-stars repos appear in the top-10-by- H .
5. **Concentrated authorship:** AI repos exhibit systematically higher push concentration; `open-webui` has one author responsible for 99.7% of pushes despite 957 PR contributors. Push concentration must be cross-validated with PR-author diversity.

Traditional metrics like stars are increasingly unreliable in isolation. Multi-dimensional measurement—community adoption, package downloads, and engineering-activity diversity—is essential for accurate OSS health assessment.

Future work includes: developing actor-level

heuristics to separate agent from human activity (commit message style, signing keys, naming patterns); extending to full-year coverage; incorporating code-review latency and issue-resolution time; sensitivity analysis on health-score weights; and longitudinal tracking of individual project health trajectories.

Acknowledgments

We thank NYU High Performance Computing for providing the Google Cloud Dataproc cluster, GH Archive for open hourly event logs, Hugging Face Hub for public model metadata, and PyPI (via Google BigQuery) for download statistics.

Data and Code Availability

The PySpark pipeline (10 analytics jobs), seed-repo list, and aggregated CSV outputs underlying every figure and table in this paper are organized in our project repository (`spark_jobs/`, `data/`, `charts/`). All raw inputs are publicly redistributable from their original sources.

References

- [1] Anthropic. Claude Code: Agentic coding in the terminal. <https://www.anthropic.com/claude-code>, 2024. Accessed May 2026.
- [2] Anysphere. Cursor: The AI code editor. <https://cursor.com/>, 2024. Accessed May 2026.
- [3] Guilherme Avelino, Leonardo Passos, Andre Hora, and Marco Tulio Valente. A novel approach for estimating truck factors. In *Proceedings of the 24th International Conference on Program Comprehension (ICPC)*, pages 1–10. IEEE, 2016.
- [4] CHAOSS Project. Community health analytics for open source software (CHAOSS): Metrics models. <https://chaoss.community/>, 2024. Accessed May 2026.
- [5] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 186–196. ACM, 2017.

- [6] Cognition AI. Devin: An AI software engineer. <https://www.cognition.ai/blog/introducing-devin>, 2024. Accessed May 2026.
- [7] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. Large language models for software engineering: Survey and open problems. *arXiv preprint arXiv:2310.03533*, 2023.
- [8] GitHub. GitHub REST/Webhook events: 2025 schema updates (`pullrequestevent.action = "merged"`; `push.distinct_size` removal). GitHub Changelog, 2025. Observed schema change between 2025 and 2026 hourly payloads; <https://github.blog/changelog/>.
- [9] Hao He, Haoqin Yang, Philipp Burckhardt, Alexandros Kapravelos, Bogdan Vasilescu, and Christian Kästner. Six million (suspected) fake stars in GitHub: A growing spiral of popularity contests, spams, and malware. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2026. arXiv:2412.13459. Identifies ~ 6 M suspected fake stars (2019–2024); large fraction of flagged repositories were subsequently removed by GitHub.